

Projet TPI - Share++

Table des matières

1	Analyse préliminaire.....	3
1.1	Rappel du projet.....	3
1.2	Environnement de développement.....	3
1.3	Objectifs.....	4
1.4	Méthode de gestion de projet.....	5
1.5	Planification initiale.....	6
2	Analyse / Conception.....	6
2.1	Concept.....	6
2.2	Stratégie de test.....	7
2.3	Risques techniques.....	8
2.4	Planification.....	9
2.5	Dossier de conception.....	9
3	Réalisation.....	10
3.1	Environnement.....	10
3.1.1	Développement sur laptop :.....	10
3.2	Structure du projet.....	10
3.3	Fonctionnalités techniques importantes.....	11
3.3.1	Workflow Build & Release automatisé.....	11
3.3.2	Logging.....	12
3.3.3	Point d'accès Wi-Fi.....	13
3.3.4	Détection et intégration Tailscale.....	14
3.3.5	Utilisation en ligne de commande (CLI).....	16
3.4	Liste des documents fournis.....	17
4	Conclusions.....	17
5	Annexes.....	18
5.1	Résumé du rapport du TPI / version succincte de la documentation.....	18
5.2	Sources – Bibliographie.....	18
5.3	Journal de travail.....	18
5.4	Manuel d'Installation.....	18
5.5	Manuel d'Utilisation.....	18
5.6	Archives du projet.....	18

1 Analyse préliminaire

1.1 Rappel du projet

Share++ est une application de bureau Windows compilée avec Flet, un framework Python basé sur Flutter. Elle produit un exécutable autonome ne nécessitant aucune installation de Python ni de navigateur sur la machine cible. L'objectif de l'application est de faciliter le partage de fichiers et de répertoires entre les appareils d'un même réseau local. Elle permet de se passer des services de cloud tiers (OneDrive, WeTransfer, Gmail, etc.) pour les transferts internes, garantissant une confidentialité totale des données et une vitesse de transfert maximale limitée uniquement par le débit du réseau local.

Ce TPI reprend le travail réalisé lors du préTPI et y ajoute cinq axes d'amélioration définis dans le cahier des charges, en plus d'apporter une correction des points laissés en suspend lors du dernier projet. Voici les axes d'améliorations dans les grandes lignes. Ils seront plus amplement détaillés à la section 1.3 :

Connectivité avancée : création d'un point d'accès Wi-Fi ad hoc

Traçabilité et audit : journalisation persistante des téléchargements

Partage à distance : détection et intégration Tailscale

Expérience utilisateur : refonte de l'interface web de téléchargement

Automatisation et CLI : interface en ligne de commande headless

1.2 Environnement de développement

Le développement a été effectué principalement sur le même laptop Linux personnel (Fedora 43), avec l'ordinateur Windows mis à disposition par le CPNV pour tester l'application et faire la validation finale des fonctionnalités. Le dépôt GitHub du préTPI a été réutilisé après archivage de son état dans une branche dédiée « prétpi ».

Détails de l'environnements :

Catégorie	Technologie / Librairie	Version	Rôle et justification
Langage de programmation	Python	3.14.3	Langage principal, choisi pour sa simplicité de développement et ses nombreuses librairies.
Framework UI	Flet (basé sur Flutter)	0.82.2	Framework de l'interface graphique de l'application, choisi pour son style, sa documentation remplie d'exemples concrets et le développement multiplateforme.
Serveur Web	aiohttp	3.13.3	Serveur HTTP asynchrone permettant de gérer les transferts de fichiers sans bloquer l'interface.
Génération QR	qrcode,	8.2 /	Génération dynamique de QR Codes pour

Code	Pillow	11.3.0	faciliter l'accès à la page de téléchargement depuis un smartphone.
Tests Unitaires	pytest / pytest- asyncio	8.4.2	Tests automatisés pour valider la logique métier.
Packaging	Flet CLI (`flet build`)	0.82.2	Outil de build natif transformant le script Python en exécutable Windows (.exe) avec les dépendances Flutter.
Installeur	Inno Setup	6.7.1	Création d'un assistant d'installation professionnel, répartissant les dépendances entre /Program Files et /Appdata et apportant les fonctionnalités des applications windows au programme.
CI / CD	Workflow Github Actions	-	Automatisation du build et du déploiement des versions (Release) à chaque mise à jour du code.
Licence	MIT	-	Licence permissive permettant l'utilisation, la modification et la distribution libre du logiciel.

1.3 Objectifs

Les objectifs principaux de Share++ sont basé sur les objectifs présents dans le cahier des charges, mais ont été reformulés comme demandé :

- **Connectivité Avancée : Création d'un Point d'Accès (Acces Point)**
 - Ce module permet de résoudre le problème de la segmentation réseau entre l'hôte et le client mobile. L'hôte créera un réseau wifi temporaire ad-hoc, afin d'intégrer la machine cliente dans le sous-réseau, afin de rendre la page de téléchargement accessible avec le QR Code.
- **Traçabilité et Audit : Historique des Téléchargements**
 - Couche de supervision du fonctionnement de l'application. Système de journalisation (logging) persistant gardant une trace de chaque requêtes réussies (status 200), ainsi que chaque requête échouées. L'historique affiche le nom du fichier, l'horodatage précis, l'adresse IP du client et tente d'identifier le type d'appareil (User-Agent parsing)
- **Partage à Distance : Détection et Intégration Tailscale**
 - L'objectif de cette partie est d'étendre la portée du partage au-delà du réseau local physique (LAN) vers un réseau maillé sécurisé (Overlay Network) par le biais d'une détection automatique de la présence du service Tailscale en cours d'exécution sur la machine hôte. Si actif, l'application récupère l'adresse IP « Tailscale » (généralement commençant par 100.x.x.x) et propose de copier un lien sécurisé accessible par tout autre appareil membre du même « Tailnet »,

permettant le partage de fichiers à travers Internet sans configuration de redirection de port (NAT traversal).

- **Expérience Utilisateur : Refonte de l'interface web de téléchargement**
 - Le candidat va procéder à une réécriture complète du frontend de la page de téléchargement servie par l'application. L'interface devient « Responsive Design » (adaptée aux mobiles, tablettes et bureaux), intègre une charte graphique moderne et offre une prévisualisation plus claire des fichiers disponibles (icônes selon le type, taille, date).
- **Automatisation et CLI : Interface en ligne de commande**
 - Ouverture de l'application aux utilisateurs avancés et aux scripts d'automatisation sera proposée.
 - Pour se faire, il procédera au développement d'un exécutable en ligne de commande (sharepp.exe) permettant de lancer un serveur headless (sans interface graphique) sur un répertoire spécifique.
 - Syntaxe cible : Sharepp /chemin/vers/dossier – port 3001 – secure
 - Fonctionnalités : le paramètre –secure active une protection par mot de passe (Basif Auth ou page de garde), et le terminal affiche un QR Code généré en ASCII/UTF-8 ainsi que les logs de connexion en temps réel.

Les corrections des défauts présents non réglés du préTPI se feront une fois ces fonctionnalités clés réalisées.

Le serveur génère une page web simple listant les fichiers disponibles, avec des liens de téléchargement directs. L'application est compilée en exécutable autonome, ne nécessitant aucune installation de Python, ou d'aucune autre dépendance sur la machine cible.

1.4 Méthode de gestion de projet

La méthode Scrum a été choisie car elle permet une livraison incrémentale adaptée à un projet de courte durée avec des objectifs bien définis.

1.5 Planification initiale

Le projet débute le 23 Avril 2026 et dure jusqu'au 20 Mai 2026. Il est découpé en 4 sprints :

1. Sprint 1 (connectivité, point d'accès, traçabilité)
2. Sprint 2 (Partage à distance et utilisation CLI)
3. Sprint 3 (Expérience utilisateur)
4. Sprint 4 (Correction préTPI et derniers correctifs)

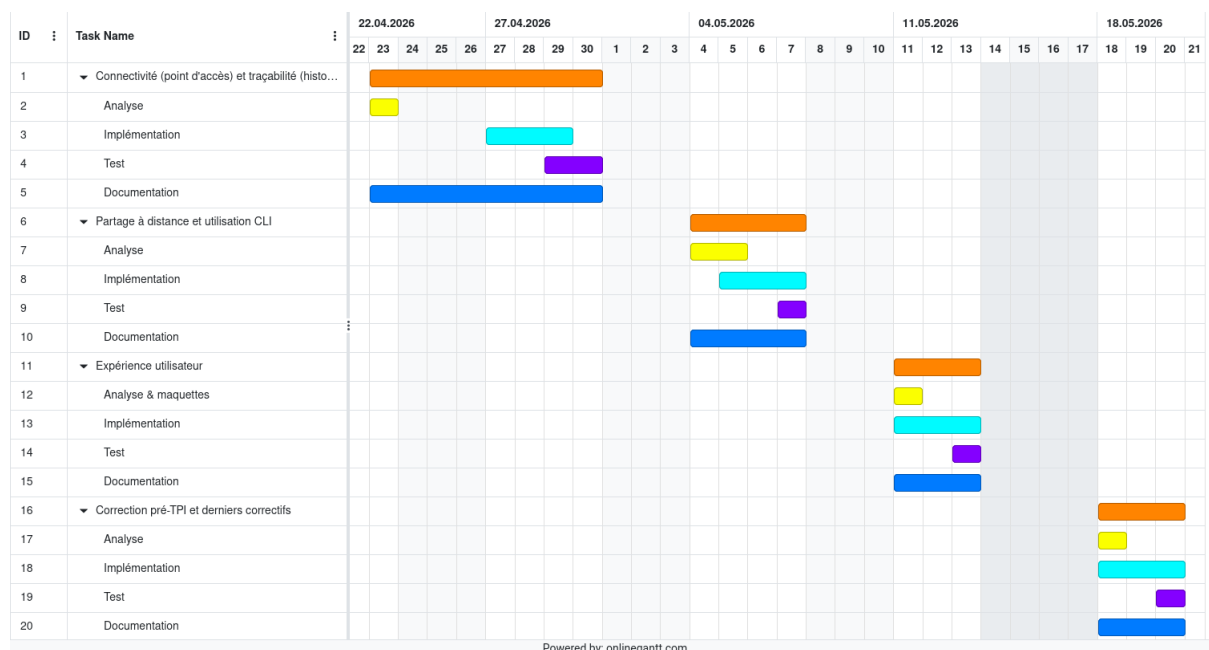


Figure 1: Gantt du planning initial

2 Analyse / Conception

2.1 Concept

Le concept est repris du projet préTPI (Share++ v1.0) et a été étendu pour inclure les nouveaux objectifs.

Le projet repose toujours sur une architecture client-serveur gérée par une application Windows native. Le code suit également toujours le modèle MVC afin de préserver la bonne organisation du code et une future maintenance de l'application :

- **Backend** : Un serveur aiohttp asynchrone gère les requêtes HTTP et le service des fichiers. Le backend sera étendu afin de supporter l'exposition de deux nouvelles interfaces réseau. La première interface est Tailscale
- **Frontend** : Une page html listant les fichiers partagés servie lorsque le partage est actif
- **Persistence** : Les paramètres de l'utilisateur, ainsi que des logs sont persistents et stockés dans des fichiers .json ou .log dédiés.
- **Application** : Une application windows native servant à gérer le serveur et ses

paramètres.

Analyse fonctionnelle :

Le cas d'utilisation principal reste inchangé : l'utilisateur sélectionne un dossier local, démarre un serveur de fichier, et les autres appareils y accèdent via un QR code ou une URL afin de télécharger les fichiers partagés. Les nouveaux cas d'utilisation sont les suivants :

- Un utilisateur souhaitant partager des fichiers avec un appareil mobile sans être sur le même réseau peut activer un point d'accès Wi-Fi ad-hoc depuis l'application, ce qui crée un sous-réseau dédié entre l'hôte et le client et permet ainsi d'accéder au partage plus facilement.
- Un utilisateur souhaitant partager des fichiers à distance (hors LAN) et disposant de Tailscale peut utiliser l'adresse Tailscale générée automatiquement par l'application.
- Un utilisateur avancé ou souhaitant automatiser le partage peut utiliser l'interface CLI pour démarrer un serveur headless sans interface graphique.
- L'historique des connexions est conservé dans un fichier de log persistant, consultable après la fermeture de l'application, avec le type d'appareil identifié via le User-Agent.

J'ai décidé de garder la licence MIT pour ce projet, car il s'agit de la licence la plus permissive au niveau de la transparence du code et sa réutilisation pour autrui.

2.2 Stratégie de test

4 types de tests sont

La stratégie de test vise à valider les nouvelles fonctionnalités tout en s'assurant que les fonctionnalités existantes n'aient pas régressé.

Types de tests :

- 1. Tests unitaires** : Vérification de la logique de parsing du User-Agent, de l'écriture et de la rotation des logs dans le fichier `sharepp.log`, et de la détection de Tailscale (présence du service, récupération de l'adresse IP).
- 2. Tests d'intégration** : Vérification que le serveur `aiohttp` expose correctement les fichiers sur les différentes interfaces réseau (LAN, hotspot ad-hoc, Tailscale), et que les logs sont correctement écrits lors des requêtes HTTP.
- 3. Tests End-to-End** : Scan du QR code depuis un smartphone connecté au point d'accès ad-hoc créé par l'application, téléchargement de fichiers et de répertoires, et vérification de la traçabilité dans le fichier log.
- 4. Tests CLI** : Vérification que le serveur headless se lance correctement avec les paramètres attendus (`--port`, `--secure`), et que le QR code ASCII s'affiche dans le terminal.

5. Tests de non-régression : Exécution de l'ensemble des tests unitaires et d'intégration du préTPI afin de garantir qu'aucune fonctionnalité existante n'a été cassée par les modifications apportées.

Moyens de mise en œuvre :

Les tests unitaires et d'intégration sont automatisés avec pytest, en continuité du projet préTPI. Les tests End-to-End sont réalisés manuellement avec le même matériel qu'au préTPI (laptop Lenovo Yoga sous Fedora 43 (Linux) comme serveur, smartphone Android Realme Neo7 comme client).

Couverture :

Les tests ne sont pas exhaustifs. Ils couvrent les fonctionnalités principales et les points de sécurité. Les données de test reprennent celles du préTPI auxquelles s'ajoutent des tests spécifiques pour les fonctionnalités de CLI et de logging.

2.3 Risques techniques

	Risque	Probabilité	Impact	Criticité	Solution
1	La création d'un point d'accès Wi-Fi ad-hoc avec python ne semble pas très courant, la fonctionnalité pourrait être fragile et rendre l'application instable	3	3	9	Utiliser une solution bien documentée
2	Régression après l'ajout de fonctionnalités avancées	1	4	4	Tester l'application après chaque release afin de s'assurer au fur et a mesure que rien ne régresse.
3	La refonte de l'interface web pourrait introduire des régressions sur la logique de téléchargement de fichiers et de répertoires.	1	4	4	Maintenir les tests d'intégration existants et les exécuter systématiquement après chaque modification du frontend.
4	L'application est trop dure à utiliser pour un utilisateur final peu	3	4	12	Refaire des maquettes et apporter des changements de design

	expérimenté				de l'application et de la page de téléchargement pour que l'expérience soit ergonomique
5	L'utilisation des fonctions actuels pour le CLI pourrait ne pas fonctionner car elles dépendent de l'interface graphique	4	4	16	Apporter des modifications aux fonctions actuels du programme afin de les rendre compatible avec l'utilisation par lignes de commandes

2.4 Planification

La planification définitive du projet reprend la structure en quatre sprints définie lors de l'analyse initiale. Le projet s'étend sur 90 heures réparties entre le 23 avril et le 20 mai 2026. Le diagramme de Gantt initial (Figure 1, section 1.5) a servi de référence, mais plusieurs ajustements ont été nécessaires en cours de réalisation.

Comparaison avec la planification initiale

Pour le premier Sprint (Connectivité avancée et traçabilité), la story portant sur la connectivité avancée par la création d'un réseau ad hoc à dû être repoussée deux fois (finie lors du Sprint 3). Le développement du point d'accès Wi-Fi ad hoc s'est heurté à deux contraintes matérielles : l'ordinateur fourni par le CPNV ne disposait d'aucune carte Wi-Fi, et le premier adaptateur USB que je me suis procuré ne supportait pas la fonctionnalité de création de réseau hébergé. Un adaptateur compatible a été trouvé le 06 mai avec l'aide de M. Wulliamoz et a permis de terminer le développement lors du deuxième Sprint, ainsi que de régler les soucis de packaging pour le Sprint 3. Cette fonctionnalité a donc été finalisée avec environ deux semaine de retard par rapport au planning initial.

Une autre source de retard a affecté plusieurs sprints : le comportement de l'application packagée (exécutable .exe généré par flet build windows) diverge de celui de l'application en environnement de développement. Ce comportement s'explique car la version packagée contient toutes les dépendences nécessaire pour tourner sur tous les environnements et affiche donc une fenêtre bloquant thread CLI pour le mode console et doit créé des fichiers, actions nécessitant des droits administrateurs spécifiques. Plusieurs régressions et problèmes spécifiques à l'exécutable (chemins de fichiers, CWD au lancement depuis le menu contextuel, attachement de console pour le mode CLI) ont nécessité des investigations non planifiées.

En réponse à ces aléas, la stratégie adoptée a été de reporter les stories non finalisées vers le sprint suivant, après analyse de faisabilité. Ainsi, la connectivité avancée et le mode CLI ont été terminés lors du Sprint 3, réduisant légèrement le temps consacré à l'expérience utilisateur (refonte de la page de téléchargement), qui constituait la dernière fonctionnalité à implémenter.

Maquettes de l'interface

Plusieurs maquettes ont été réalisées pour ce TPI, correspondant aux évolutions de l'interface par rapport au préTPI :

Maquette de l'application : Bouton démarrage/arrêt du serveur toujours en évidence, ExpansionTile pour chaque code QR (cachés par défaut), permettant de ne pas inonder l'interface avec des code QR, ajout du paramétrage de la configuration du hotspot Wi-Fi ad hoc (SSID et mot de passe) reprenant la logique d'implémentation de la protection par mot de passe (via un ft.chip qui s'étend)

Maquette de la page de téléchargement : reflète les fonctionnalités à ajouter telles que décrites dans le cahier des charges (interface responsive, icônes par type de fichier, affichage de la taille des fichiers, bulk-downloads, refont de l'interface de connexion).



Figure 2: Interface de l'application (configuration du partage)

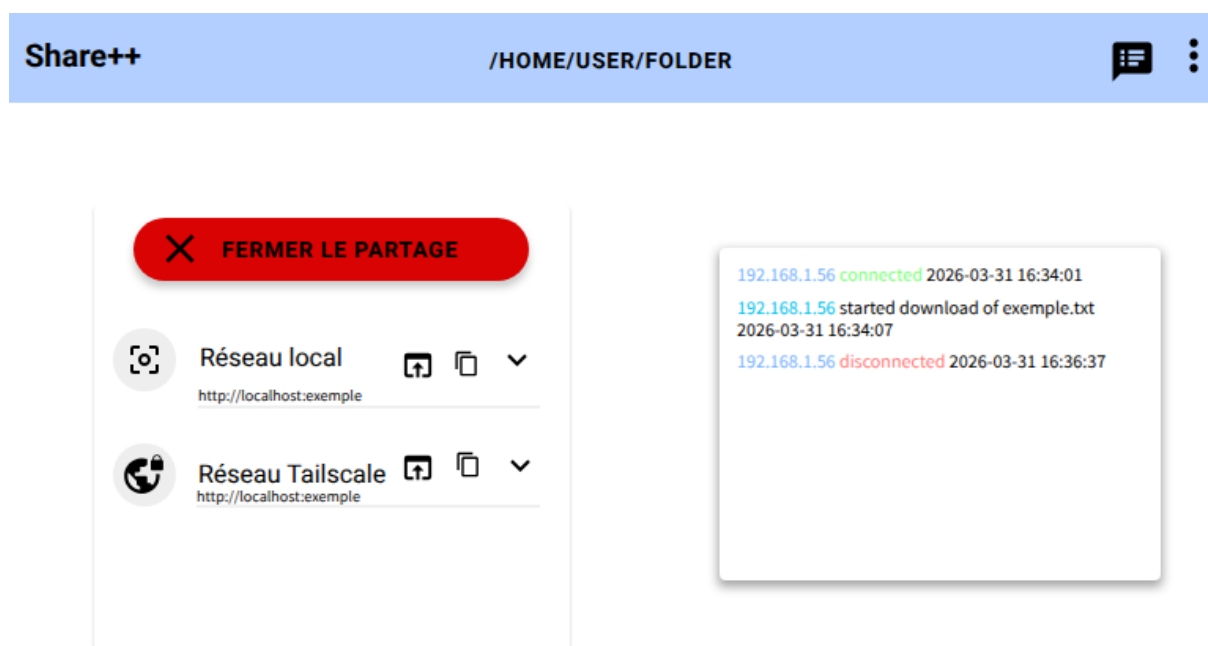


Figure 3: Interface de l'application (serveur ouvert sans hotspot)

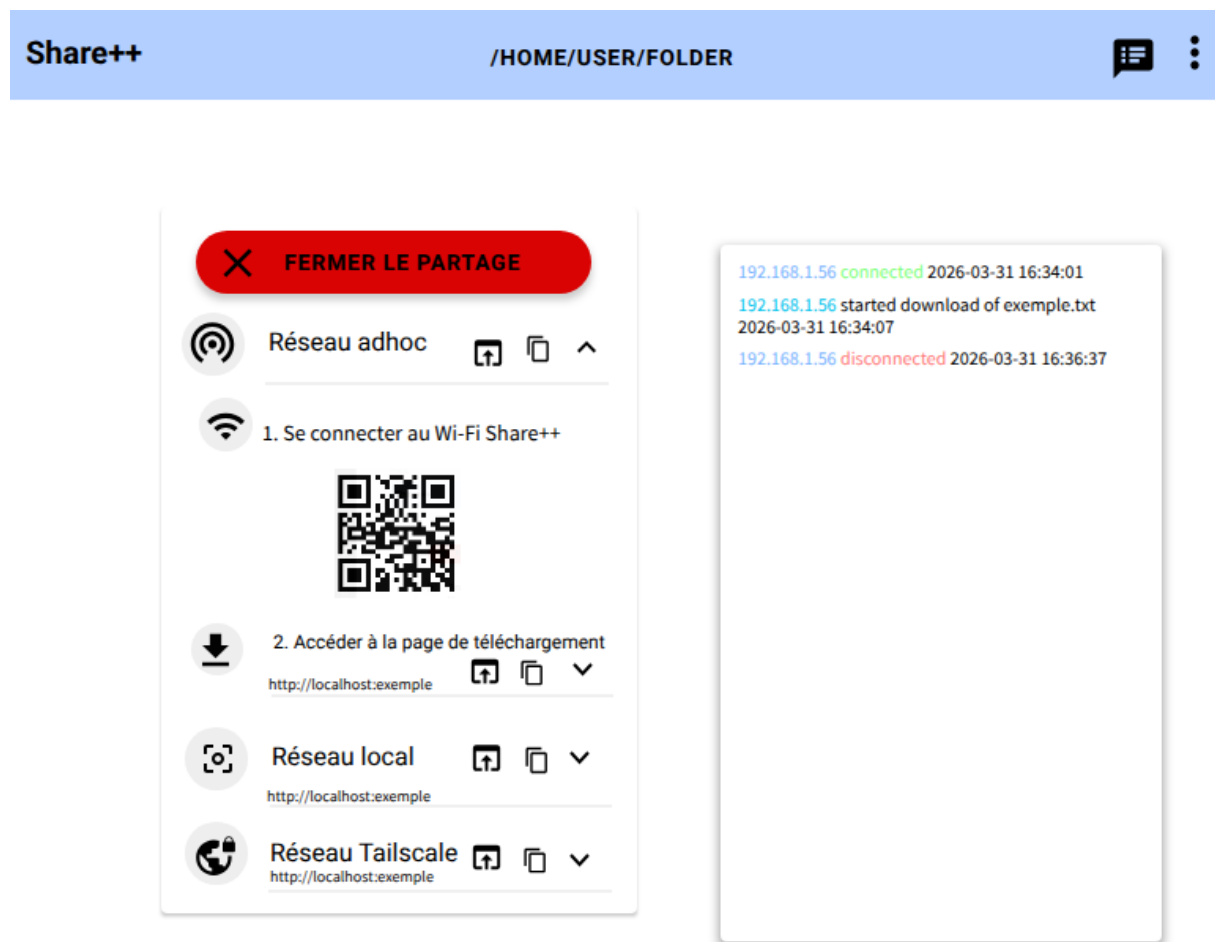


Figure 4: Interface de l'application (serveur ouvert avec hotspot)



Figure 5: Interface de l'historique de téléchargement

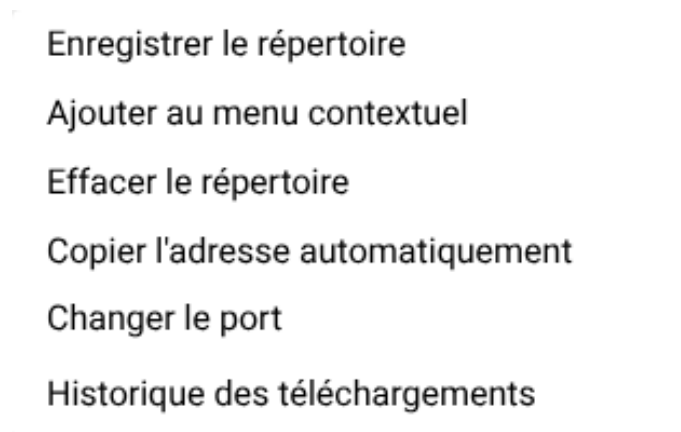


Figure 6: Interface des paramètres

2.5 Dossier de conception

Choix matériels et logiciels

Les choix technologiques sont détaillés dans le tableau de la section 1.2.

L'environnement de développement principal reste le laptop personnel Lenovo Yoga sous Fedora 43 (Linux). Les tests et la validation finale sont effectués sur le poste

Windows 11 mis à disposition par le CPNV, avec un smartphone Android realme Neo7 comme client de téléchargement.

L'exécutable produit par build de flet-cli est un binaire Windows autonome embarquant le runtime Flutter et le runtime Python. Contrairement à une application Electron qui embarque Chromium, ou à une application web progressive, l'interface graphique de Share++ est rendue nativement par le moteur Flutter (Skia/Impeller), sans navigateur ni WebView. Il serait donc inexact de qualifier cette application d'application web ou d'application hybride au sens courant du terme : il s'agit d'une application de bureau compilée dont le code UI est écrit en Python via l'abstraction Flet. Cette distinction a eu des implications concrètes sur le build : le processus d'initialisation Flutter démarre avant Python, ce qui a rendu impossible l'intégration du mode CLI dans le même exécutable (voir section 3.3.5).

Architecture de l'application

Le projet conserve l'architecture **MVC** (Model, View, Controller) établie lors du préTPI. Cette séparation garantit que chaque composant a une responsabilité unique, cela facilite la reprise du projet plus tard et la lisibilité du code.

Le découpage est le suivant :

Model (models/model.py) : gestion de la persistance des paramètres utilisateur (settings.json), du système de journalisation persistante (sharepp.log), et de toutes les opérations sur les données (hachage de mot de passe, toggles de configuration).
View (views/view.py, views/download_view.py) : interface de l'application graphique Flet et génération de la page HTML de téléchargement servie par aiohttp.
Controller : décomposé en quatre modules distincts pour éviter la monolithisation du contrôleur principal :

app_controller.py : orchestre les interactions entre l'UI Flet, le modèle et les contrôleurs spécialisés.

server_controller.py : logique du serveur aiohttp partagée entre le mode GUI et le mode CLI.

network_controller.py : détection des interfaces réseau disponibles (LAN, Tailscale, hotspot).

hotspot_controller.py : création et gestion du point d'accès Wi-Fi via netsh.

Diagramme de classe

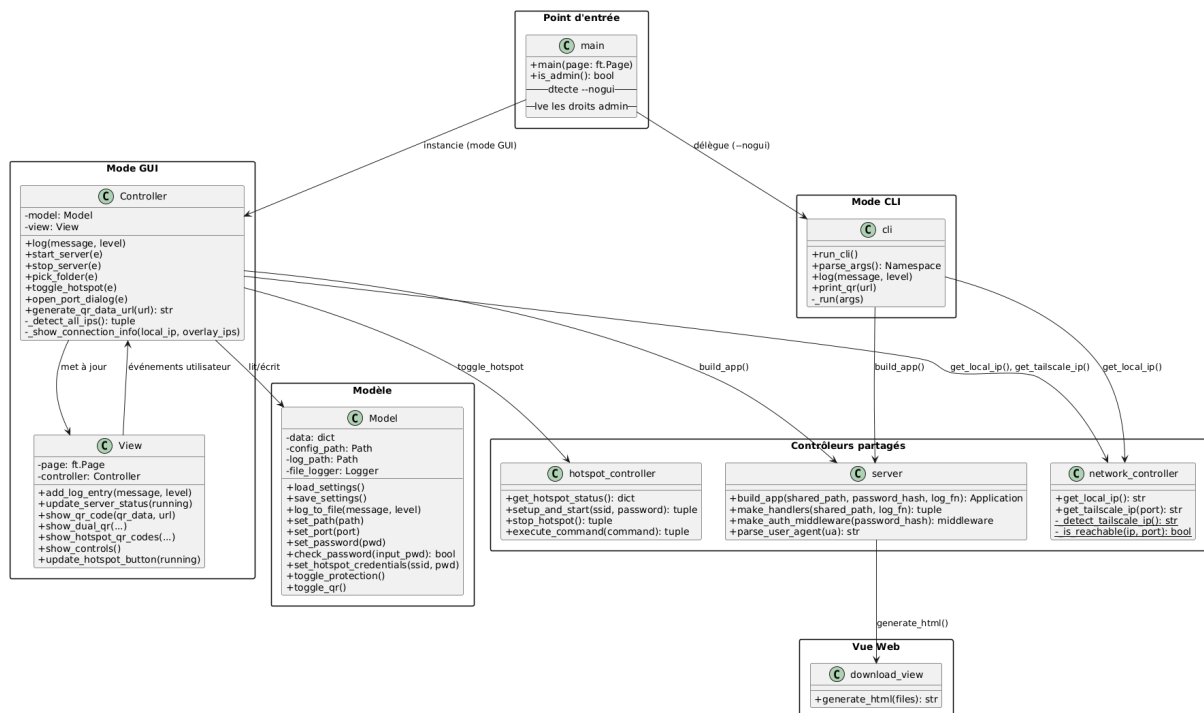


Figure 7: Diagramme de classe (UML) représentant l'architecture du projet

Modèle de persistance

Les données persistantes de l'application sont stockées dans deux fichiers situés dans %APPDATA%\SharePlusPlus\ :

settings.json : persistence des paramètres utilisateurs:

```
json{
  "selected_path": "/path/to/...",
  "port": 8080,
  "is_protected": false,
  "password": "",
  "logs_visible": true,
  "qr_enabled": true,
  "remember_path": true,
  "copy_to_clipboard": false,
  "context_menu_enabled": false,
  "hotspot_enabled": false,
  "hotspot_ssid": "SharePlusPlus",
  "hotspot_password": "sharepp1"
}
```

sharepp.log : journal persistant des événements, avec rotation automatique à 1 Mo (3 fichiers conservés) :

```
2026-04-27 12:06:43 [INFO]   Serveur ouvert sur le port 8080
2026-04-27 12:06:54 [INFO]   Connexion de 10.227.140.6 [PC (Linux)]
2026-04-27 12:06:54 [WARNING] 10.227.140.6 [PC (Linux)] télécharge fichier.pdf
2026-04-27 12:07:01 [ERROR]   Serveur fermé
```

3 Réalisation

3.1 Environnement

J'ai utilisé le même environnement pour ce travail que pour mon projet préTPI, c'est à dire que j'ai utilisé un laptop Linux personnel pour le développement, puis j'ai testé l'application depuis l'ordinateur sous Windows 11 mis à notre disposition par le CPNV. Tester l'application sur un poste Windows est important comme je développe sur un environnement techniquement non supporté par mon application (application windows développée sur un poste Linux). Il est donc possible de valider ainsi les fonctionnalités implémentées.

J'ai repris le dépôt Github du projet préTPI (<https://github.com/NathanFilipowicz/Sharepp>) et ai continué de l'utiliser pour ce projet, après avoir archivé l'état du dépôt dans une branche 'préTPI' dédiée. J'ai également créé des répertoires afin de ne pas garder tous les documents dans un seul répertoire /docs comme lors de mon dernier projet.

3.1.1 Développement sur laptop :

J'ai principalement utilisé mon laptop personnel pour réaliser ce travail. J'ai conservé la logique de séparation de code de mon projet préTPI. Les fichiers de code suivent la logique MVC (Model-View-Contrôleur) traduite ainsi dans mon projet :

- /src/controllers, code dédié à la logique métier de l'application. Les fonctionnalités, l'orchestration
- /src/models, code dédié à la persistance des données
- /src/views, code dédié à l'interface graphique de l'application et de la page de téléchargement

ainsi qu'un main.py servant de point d'entrée de l'application flet, et détecte désormais le niveau de privilège afin de pouvoir s'élever afin que la fonctionnalité de création d'access point fonctionne

Développement sur ordinateur CPNV :

J'ai eu besoin de migrer le projet sur l'ordinateur fixe du CPNV afin de pouvoir intégrer la fonctionnalité spécifique windows du bouton contextuel de l'explorateur de fichiers. Pour ceci, j'ai cloné le projet depuis mon dépôt github afin de pouvoir travailler sur l'environnement et pouvoir tester l'application directement. J'ai aussi utilisé cet environnement pour créer l'installateur InnoSetup pour le build final windows de l'application.

3.2 Structure du projet

La structure est également représentée plus succinctement dans le README du projet. L'architecture choisie est l'architecture MVC (Model, View, Controller) me permettant de séparer la logique de mon code.

```
Sharepp/
├── .github/
│   └── workflows/
│       └── build-release.yml
├── docs/
│   ├── Dossier_de_projet.odt
│   ├── Journal_de_travail.ods
│   └── Journal_de_test.pdf
```




3.3 Fonctionnalités techniques importantes

3.3.1 Workflow Build & Release automatisé

Un workflow GitHub Actions (.github/workflows/build-release.yml) automatise entièrement le build windows de l'application, ainsi que le packaging et la publication de l'installateur Windows sous forme de release versionnée sur le dépôt Github. Il se déclenche à chaque push s'accompagnant d'un tag de version -v*.

Etapes du workflow

1. Récupération du code du dépôt et configuration de Python 3.13.
2. Extraction de la version depuis le tag Git (v1.0.0 → 1.0.0).
3. Installation des dépendances Python depuis requirements.txt.
4. Installation du SDK Flutter.
5. Build de l'exécutable Windows via flet build windows.
6. Installation d'InnoSetup avec Chocolatey.
7. Mise à jour dynamique du numéro de version dans setup.iss.
8. Génération de l'installateur .exe via **ISCC.exe** (Inno Setup Command-Line Compiler).

9. Build de l'exécutable CLI sharepp-cli.exe via PyInstaller (--onefile --console).
10. Création de la release GitHub et upload de l'installateur.

Ce pipeline garantit la reproductibilité des releases, leur versionnement et permet aux utilisateurs de pouvoir facilement et rapidement accéder à une version à jour et facilement installable de l'application.

3.3.2 Logging

- Utilisation de la librairie « logging » de la librairie standard de python.
- Écriture dans un fichier sharepp.log avec rotation automatique pour éviter les fichiers logs trop volumineux.
- Ajout d'une nouvelle donnée dans les logs : Le type d'appareil utilisé pour accéder au partage (récupéré grâce au User-Agent parsing).
- Intégration dans le système précédent de logs sans trop de réécriture

L'application disposait déjà d'un affichage des logs en temps réel dans son interface dès sa version 1.0 (préTPI), mais ces informations étaient perdues à chaque fermeture. Pour répondre à l'objectif de traçabilité du cahier des charges, un système de journalisation persistant a été ajouté au modèle, permettant de conserver un historique des logs précédemment affichés qu'en temps réel. Ceci permet à l'utilisateur de pouvoir consulter les logs a posteriori, sans avoir à ouvrir l'application. L'utilisateur aura également droit à d'avantage d'informations sur le type d'appareil (« [Mobile[Android]] », « [PC[Windows]] »...) qui s'est connecté à son serveur.

La librairie logging de python repose sur trois concepts : le **Logger**, qui reçoit les messages et filtre par niveau de sévérité ; le **Handler**, qui dirige les messages vers leur destination (ici un fichier sharepp.log), et le **Formatter**, qui définit la mise en forme de chaque entrée.

```
# Persistent log file path
self.log_path = self.config_dir / "sharepp.log"
self._setup_file_logger()

def _setup_file_logger(self):
    # Identify log file (create if not exists)
    self.file_logger = logging.getLogger("sharepp")
    # Accept logs of all severity levels (DEBUG == lowest priority)
    self.file_logger.setLevel(logging.DEBUG)

    if not self.file_logger.handlers:
        handler = RotatingFileHandler(
            self.log_path, maxBytes=1_000_000, backupCount=3, encoding="utf-8"
        )
        formatter = logging.Formatter(
            "%(asctime)s [%(levelname)s] %(message)s",
            datefmt="%Y-%m-%d %H:%M:%S"
        )
        handler.setFormatter(formatter)
        self.file_logger.addHandler(handler)
```

Emplacement du fichier de log

Le fichier logs est enregistré dans le répertoire Appdata de Windows, aux côtés de settings.json, ce répertoire est le répertoire conseillé pour les documents dont il faut le droit d'accès en écriture.

Retrouver le fichier :

%APPDATA% -> Roaming\SharePlusPlus\sharepp.log

Format des logs

2026-04-27 12:06:43 [INFO] Serveur ouvert sur le port 8080

2026-04-27 12:06:54 [INFO] Connexion de 10.227.140.6 [PC (Linux)]

2026-04-27 12:06:54 [WARNING] 10.227.140.6 [PC (Linux)] téléchargé favicon.ico

2026-04-27 12:07:01 [ERROR] Serveur fermé

Rotation automatique

Pour ne pas réécrire continuellement sur le même fichier log, un RotatingFileHandler a été utilisé, servant à créer un nouveau fichier log après que celui-ci ait dépassé 1 Mo, et ne conserve que trois versions de fichiers logs. Comme toutes les logs affichés sur l'application sont enregistrés automatiquement, cela garantit que l'espace disque autorisé pour les logs soit limité à 4 Mo.

Intégration avec le contrôleur

La méthode Controller.log(message, niveau) appelle désormais model.log_to_file() pour la persistance du log, et continue de l'afficher dans l'interface Flet en même temps.

Identification des appareils (User-Agent Parsing)

Chaque entrée de log issue d'une connexion entrante, inclut une identification de l'appareil client déduite par le header HTTP User-Agent. La méthode parse_user_agent() du contrôleur récupère le type d'appareil (Mobile, Tablette, PC) et le système d'exploitation (Android, iOS, Windows, Linux, macOS) de chaque requêtes de téléchargement entrante (pas pour les logs informatifs de type Password protection enabled).

Exemples de sorties :

Mobile (Android)

PC (Windows)

Appareil inconnu

Page d'historique des téléchargements

Le fichier sharepp.log sert également à récupérer les derniers téléchargements directement depuis l'application. Un bouton ajouté au PopMenuButton « Voir l'historique des téléchargements » affiche une fenêtre présentant les derniers noms de fichiers partagés, incluant leur métadonnées et la possibilité d'ouvrir le fichier dans l'explorateur windows s'ils sont toujours présent et au même emplacement.

Parsing du fichier log

La méthode get_download_history() ajouté au modèle lit le fichier sharepp.log et en extrait les entrées de téléchargement via l'expression régulière suivante :

(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}) \[WARNING\] ([\d\.]+) \[(.+?)\] téléchargé '(.+)'

Cette expression régulière cible les entrées de niveau [WARNING] correspondant à un téléchargement, et capture quatre groupes : l'horodatage, l'adresse IP du client, le type d'appareil et le nom du fichier. Les résultats sont retournés dans l'ordre inverse afin d'afficher le téléchargement le plus récent en premier.

Interface de l'historique des téléchargements

Chaque entrée de l'historique est affichée sous forme d'une carte avec le nom du fichier en gras, l'horodatage, l'adresse IP et le type d'appareil. Un bouton d'ouverture dans l'Explorateur Windows est disponible pour chaque entrée : il est actif uniquement si le dossier partagé est encore sélectionné dans l'application et si le fichier existe toujours à son emplacement d'origine. Si ces conditions ne sont pas remplies, le bouton est désactivé. L'ouverture se fait via `subprocess.run(["explorer", "/select,", filepath])`, ce qui ouvre l'Explorateur avec le fichier sélectionné.

3.3.3 Point d'accès Wi-Fi

Le cas d'usage principal de Share++ consiste à partager des fichiers entre un ordinateur de bureau et un smartphone. Cependant, il est commun que le smartphone et l'ordinateur ne partagent pas le même sous-réseau, si le smartphone est connecté en 5g par exemple. Dans ce contexte, le client mobile n'est pas en mesure d'accéder à la page de téléchargement générée par le serveur aiohttp.

La solution retenue est de permettre à l'hôte de créer lui-même un réseau Wi-Fi ad hoc temporaire. Le smartphone se connecte à ce réseau, rejoignant ainsi le sous-réseau de l'hôte et rendant la page accessible.

Implémentation

La logique de création, démarrage et arrêt du point d'accès est inscrite dans le fichier `src/controllers/hotspot_controller.py`, afin de respecter l'architecture MVC du projet.

Fonctions ajoutées

- `get_hotspot_status()` : lance la commande ``netsh wlan show hostednetwork`` et détecte si le point d'accès est actif, en traitant la sortie de la commande en français et en anglais (bilingue, car le retour de la commande sur la version francophone de windows est en français).
- `setup_and_start(ssid, password)` : valide le mot de passe (minimum 8 caractères, exigence de tout mot de passe Wi-Fi WPA2), exécute la commande de configuration puis démarre le réseau.
- `stop_hotspot()` : arrête le point d'accès.
- `execute_command(command)` : wrapper autour de `subprocess.run` avec le flag `CREATE_NO_WINDOW (0x08000000)` pour éviter les flashes de terminal Windows. Implémenté avec l'aide de Gemini (voir journal de travail).
- `ensure_admin()` : vérifie les droits administrateur et relance l'application avec élévation si nécessaire.

Intégration dans l'interface

La hotspot card indépendante présente dans une version antérieure a été supprimée pour une intégration dans la card principale de configuration du partage. Deux contrôles ont été ajoutés :

Un bouton "Créer un hotspot Wi-Fi" permet d'activer la fonctionnalité de manière persistante dans settings.json. Ce bouton est affiché sur tous les systèmes d'exploitation mais est désactivé sur les systèmes non-Windows.

Un bouton "Configurer le réseau Wi-Fi" ouvre un AlertDialog contenant les champs SSID et mot de passe. Conformément à l'architecture MVC, la création et l'affichage de ce dialog sont entièrement gérés dans view.py le contrôleur se limite à valider et persister les données via save_hotspot_credentials().

Flux de l'expérience utilisateur

Lorsque l'utilisateur active le hotspot depuis l'interface :

1. Share++ crée le réseau avec le SSID et le mot de passe configurés.
2. Une bannière d'instruction apparaît au-dessus des tuiles de connexion, invitant l'utilisateur à se connecter d'abord au réseau Wi-Fi avant de scanner l'URL.
3. Un QR code Wi-Fi est disponible dans une ExpansionTile dédiée, encodant la chaîne WIFI:T:WPA;S:{ssid};P:{pwd};H;; qui permet de se connecter automatiquement en le scannant depuis un smartphone.
4. L'adresse IPv4 de la machine servant le serveur de fichier : `http://192.168.137.1:{port}` apparaît comme option de connexion supplémentaire avec un QR code de connexion aux côtés des adresses de l'ip locale et de l'ip Tailscale.
5. Une fois connecté au hotspot, l'utilisateur peut scanner le QR code issu du hotspot adhoc pour accéder aux fichiers. A l'arrêt du serveur de fichier, le réseau adhoc se ferme automatiquement.

Problèmes rencontrés

L'utilisation de netsh (interpréteur de commandes réseau Windows) pour la création de l'accès point oblige à tester la fonctionnalité sur une machine Windows. Le problème étant que l'ordinateur de test mis à ma disposition pour ce travail n'avait pas de carte Wi-Fi et ne pouvait donc pas afficher les commandes netsh recherchées. J'ai donc dû me procurer une clé USB Wi-Fi afin de pouvoir tester et corriger l'implémentation.

Même après avoir obtenu la clé Wi-Fi TP-Link et installé les drivers, je me suis rendu compte que la clé ne supportait pas la fonctionnalité 'réseau hébergé' (netsh wlan show drivers → Réseau hébergé pris en charge : non), prérequis nécessaire à la commande netsh wlan hostednetwork.

Le Mercredi 06 mai, M. Wuliamoz et moi avons testé plusieurs adaptateurs USB ainsi qu'un laptop windows, jusqu'à ce que nous ayons trouvé un adaptateur USB qui supporte la fonctionnalité me permettant alors de poursuivre l'implémentation avec un peu de retard.

3.3.4 Détection et intégration Tailscale

Contexte et objectif

Share++ est conçu pour fonctionner sur un réseau local. Cependant, il est fréquent qu'un utilisateur souhaite partager des fichiers avec un appareil situé hors de son réseau (comme avec un smartphone en 5g par exemple), sans pour autant avoir

configuré manuellement une redirection de port sur son routeur. Tailscale est un service de réseau maillé sécurisé basé sur WireGuard qui résout exactement ce problème : chaque appareil du sous-réseau Tailscale appelé "Tailnet" reçoit une adresse IP fixe de plage 100.x.x.x et peut communiquer avec les autres machines du Tailnet, pourvu qu'ils aient un accès à internet.

L'objectif est de détecter automatiquement la présence du service Tailscale sur la machine hôte au moment du démarrage du serveur, et de proposer un lien de téléchargement supplémentaire via cette interface, sans aucune configuration manuelle de la part de l'utilisateur.

Architecture network_controller.py

La logique de détection réseau a été écrite dans un fichier dédié : `src/controllers/network_controller.py`, conformément à l'architecture MVC du projet. Ce contrôleur expose trois fonctions publiques :

- `get_local_ip()` : détecte l'adresse IP du réseau local (précédemment directement présent dans `app_controller.py`).
- `get_tailscale_ip(port)` : détecte l'adresse IP Tailscale et valide son accessibilité.
- `get_hotspot_ip()` : détecte l'adresse IP de l'interface du point d'accès Wi-Fi ad hoc (voir section 3.3.3).

Détection de l'adresse IP Tailscale

Pour détecter la présence de Tailscale sur la machine de l'utilisateur, on lance une commande tailscale CLI permettant d'afficher l'ip Tailscale de la machine.

La détection se déroule en deux étapes. D'abord, `tailscale status` est exécuté via `subprocess.run()` avec un timeout de 3 secondes. Son code de retour détermine si le service est actif. Si le service est inactif, aucun changement dans l'application ne sera observé.

Si le service est actif, `tailscale ip --4` est exécuté et récupère l'adresse IPv4 de Tailscale. L'adresse est aussi validée par une expression régulière garantissant qu'elle appartient bien à la plage Tailscale.

Validation de l'adresse IP Tailscale

Pour s'assurer que la connexion à la page de téléchargement fonctionne correctement via l'IP Tailscale, une méthode privée `_is_reachable()` a été implémentée, lançant une tentative de connexion via la librairie socket afin de pouvoir proposer la fonctionnalité à l'utilisateur, que si elle fonctionne vraiment.

```
def _is_reachable(ip, port, timeout: float = 3.0):
    try:
        with socket.create_connection((ip, port), timeout=timeout):
            return True
    except OSError:
        return False

def _detect_tailscale_ip():
    # Is tailscale active: "tailscale status" output
    try:
        status = subprocess.run(
            ["tailscale", "status"],
            capture_output=True, text=True, timeout=3
        )
        if status.returncode != 0:
            return None

        # get tailscale ip for this machine in IPv4
        result = subprocess.run(
            ["tailscale", "ip", "--4"],
            capture_output=True, text=True, timeout=3
        )
        ip = result.stdout.strip()
        if ip and re.match(r"^100\.(6[4-9]|[7-9]\d|1[0-1]\d|12[0-7])\.\d{1,3}\.\d{1,3}$", ip):
            return ip

    except Exception as e:
        print(f"Error during Tailscale ip detection: {e}")

    return None
```

Alternative envisagée

En connaissant la plage standard d'attribution d'adresses IPV4 de Tailscale, il aurait été possible de récupérer les adresses IPV4 de la machine via la librairie python standard 'socket'. Cela aurait eu l'avantage de fonctionner en l'absence de tailscale dans le PATH. Cette méthode n'a pas été retenue car une adresse pourrait commencer par 100. sans venir de Tailscale, rendant la détection moins fiable. La méthode CLI ayant fonctionné directement, cette alternative n'a pas été poursuivie.

Validation de l'accessibilité

La fonction `get_tailscale_ip(port)` valide l'adresse IPV4 Tailscale détectée via une tentative de connexion TCP (`socket.create_connection`) sur le port du serveur. Cela sert à valider le bon fonctionnement du lien de téléchargement hors-site avant de le proposer à l'utilisateur.

get_local_ip()

La fonction `get_local_ip()` détecte l'IP locale via une connexion UDP factice vers 8.8.8.8. L'implémentation reprend du code directement de la documentation officielle disponible dans la section sources de ce document.

Intégration dans le flux de démarrage du serveur

La détection est déclenchée automatiquement à chaque démarrage du serveur de fichier, dans la méthode `start_server()` du contrôleur. Elle s'exécute dans un thread

séparé via `asyncio.to_thread()` afin de ne pas bloquer la boucle d'événements Flet pendant le scan réseau (`subprocess.run()` est bloquant) :

```
local_ip, tailscale_ip = await asyncio.to_thread(self._detect_all_ips)
await self._show_connection_info(local_ip, tailscale_ip)
```

La méthode `_show_connection_info()` génère ensuite les QR codes et met à jour l'interface selon les IPs disponibles :

Si seul le LAN est disponible : un seul `ExpansionTile` QR est affiché (comportement inchangé par rapport à la version préTPI).

Si Tailscale est détecté : deux `ExpansionTiles` indépendants sont affichés, un pour l'adresse LAN et un pour l'adresse Tailscale.

Aucune donnée liée à Tailscale n'est inscrite dans le fichier `settings.json`. La détection est donc entièrement en mémoire et se relance à chaque session afin de rendre l'expérience utilisateur plus fiable.

Interface utilisateur

L'affichage des QR codes a alors été refait pour épurer l'interface utilisateur. Les QR codes ne s'affichent plus en grand au démarrage du serveur mais sont masqués par défaut dans des composants `ft.ExpansionTile`. Chaque tile expose sur lui de manière toujours visible :

L'URL du partage sélectionnable.

Un bouton d'ouverture dans le navigateur.

Un bouton de copie dans le presse-papier.

Le bouton d'arrêt du serveur reste toujours visible au-dessus des `ExpansionTiles`.

3.3.5 Utilisation en ligne de commande (CLI)

Contexte et objectif

Share++ est conçu comme une application graphique Windows. Cependant, certains power users apprécieront d'avoir la possibilité d'ouvrir un serveur de fichier sécurisé en une ligne de commande directement dans leur terminal. L'objectif de cette fonctionnalité de permettre à l'utilisateur d'utiliser **l'outil** Share++ sans passer par son interface graphique, en préservant les fonctionnalités principales telle que le qr code, la protection par mot de passe, les logs.

Analyse des variantes

Deux approches principales ont été envisagées pour l'implémentation de cette fonctionnalité :

- Variante 1 Exécutable unique : `main.py` détecte la présence de l'argument `--nogui` au démarrage et délègue l'exécution à un module CLI dédié, sans jamais initialiser Flet.
- Variante 2 Exécutable séparé : un second exécutable `sharepp-cli.exe` est buildé indépendamment, sans aucune dépendance Flet, produisant un binaire plus léger.

Méthode retenue

La deuxième variante a été poursuivie après avoir rencontré un problème bloquant avec la première variante. En effet, il n'était simplement pas possible d'appeler `sharepp.exe build` sans qu'une fenêtre vide ne s'affiche, bloquant par la même occasion le thread du module CLI. Pour se faire, une étape de build de l'exécutable via `pyinstaller` a été ajoutée au workflow `build-release`. La version CLI est alors proposée en dessous de la release de l'application GUI.

3.3.6 Refonte de la page de téléchargement

Contexte et objectif

La page de téléchargement servie par `aiohttp` affiche les fichiers servis par Share++ aux appareils client. Dans la version préTPI, cette page se limitait à une liste de fichiers avec un bouton de téléchargement par entrée, sans métadonnées, sans responsive, et avec une interface d'authentification reposant sur le mécanisme natif `WWW-Authenticate: Basic`, qui affichait un popup navigateur natif avec deux champs (utilisateur et mot de passe), la solution n'était pas idéale car Share++ n'intègre pas la notion d'utilisateurs.

L'objectif de ce sprint était de réécrire entièrement cette interface pour la rendre plus pratique à l'utilisation en intégrant les points suivants : adapter l'interface aux appareils mobiles (responsive design), plus informative (ajout des métadonnées des fichiers et de styles).

Architecture des fichiers

La logique de présentation est désormais répartie sur trois fichiers dans `src/views/` :

```
└─ download_view.py    # génération du HTML squelette
└─ static/
    └─ download.css    # styles (page liste + page auth)
    └─ download.js     # sélection multiple et bulk download
```

`server_controller.py` expose ces fichiers statiques via `app.router.add_static("/static", _STATIC_DIR)`. La route `/static/` est enregistrée avant `/filename` pour éviter toute ambiguïté avec les fichiers partagés.

Page de téléchargement

La fonction `generate_html(files, shared_path)` reçoit désormais `shared_path` en plus de la liste de fichiers, afin de lire la taille et la date de modification réelles de chaque fichier via `os.path.getsize` et `os.path.getmtime`. Pour chaque fichier, la page affiche une icône colorée selon le type, le nom du fichier, sa taille en format lisible (o / Ko / Mo / Go) et sa date de dernière modification.

Les icônes SVG sont issues de Feather Icons (licence MIT). Elles sont définies dans une liste de dictionnaires `FILE_TYPES`, chaque entrée contenant la liste des extensions associées, une couleur hexadécimale et les paths SVG correspondants. Deux fallbacks existent : `ICON_FOLDER` pour les répertoires et `ICON_UNKNOWN` pour les extensions non reconnues.

Les fichiers cachés (nom commençant par `.`) reçoivent la classe CSS `hidden-file` : opacité réduite et italique, pour les signaler visuellement sans les masquer.

Sélection multiple (bulk download)

Une checkbox placée à gauche de chaque carte permet la sélection multiple. Un bouton de téléchargement groupé, masqué par défaut, apparaît dès qu'au moins une checkbox est cochée et indique le nombre de fichiers sélectionnés. Les fichiers se

téléchargement alors l'un après l'autre avec un intervalle de 400 ms entre chaque fichier afin de prévenir des restrictions du navigateur. La logique de sélection et de téléchargement est entièrement gérée dans `download.js`.

Page d'authentification

Le mécanisme WWW-Authenticate: Basic a été remplacé par une page HTML servie sur la route `/password`. Cette page ne contient qu'un champ mot de passe, sans champ utilisateur. En cas d'erreur, un message s'affiche dans la même page sans rechargement complet. Le CSS de cette page est inclus dans `download.css` sous une section dédiée appliquée via `body.auth-page`, pour ne pas interférer avec les styles de la page liste.

Authentification par cookie signé

Le middleware `make_auth_middleware` a été mis à jour pour vérifier un cookie de session signé avec HMAC-SHA256 au lieu du header `Authorization: Basic`. Les routes `/password` et `/static/*` sont exemptées afin que le CSS soit accessible avant l'authentification. Lorsque le mot de passe soumis est correct, un cookie `sharepp_session` est posé (`httponly`, `SameSite=Strict`), signé avec une clé aléatoire générée au démarrage (`os.urandom(32)`). Ce mécanisme invalide automatiquement toutes les sessions à chaque redémarrage de Share++, sans nécessiter de base de données de sessions.

3.4 Liste des documents fournis

Les éléments suivants constituent la livraison finale du projet Share++ TPI. Tous les documents sont en version 1.0.

Document	Format	Description
Dossier de projet	PDF / ODT	Document principal comprenant l'analyse, la conception et la réalisation
Journal de travail	ODS	Rapports journaliers des activités, succès, échecs et aides externes
Journal de test	PDF / ODS	Plan de test et résultats d'exécution des cas de test
Maquette application	PNG	Maquette de l'interface graphique Flet (version TPI)
Maquette page de téléchargement	PNG	Maquette de la page web de téléchargement refaite
Schéma réseau	PNG / drawio	Schéma de l'architecture réseau client-serveur
Code source	GitHub	Dépôt public <code>NathanFilipowitz/Sharepp</code>
Installateur Windows	EXE	<code>SharePlusPlus_Setup_1.0.0.exe</code> , assistant d'installation GUI
Exécutible CLI	EXE	<code>sharepp-cli.exe</code> , version headless en ligne de commande

4 Conclusions

4.1 Objectifs atteints

Les cinq axes d'amélioration définis dans le cahier des charges ont été implémentés :

- **Traçabilité et audit** : le système de journalisation persistante (`sharepp.log`) avec rotation automatique et identification des appareils par User-Agent parsing est pleinement fonctionnel et testé.
- **Partage à distance** : la détection automatique de Tailscale est implémentée et intégrée au flux de l'utilisation de l'application. L'adresse Tailscale est proposée à l'utilisateur uniquement si le service est actif **et** l'adresse IP joignable.
- **Connectivité avancée** : le point d'accès Wi-Fi ad hoc via `netsh wlan hostednetwork` est fonctionnel sur une machine Windows compatible (disposant d'une carte Wi-Fi supportant l'hébergement réseau). La fonctionnalité est désactivée proprement sur les systèmes non supportés.
- **Automatisation CLI** : `sharepp-cli.exe` est un exécutable autonome généré via PyInstaller, distribué dans la même release GitHub que l'installateur GUI.
- **Expérience utilisateur** : la page de téléchargement a été refaite. Elle intègre désormais le responsive, des icônes par type de fichier, l'affichage de la taille du fichier, la date de dernière modification du fichier, la sélection multiple avec téléchargement groupé (bulk download), et une page d'authentification HTML remplaçant le popup natif du navigateur.

Workflow CI/CD: Un workflow GitHub Actions génère et publie automatiquement les deux artefacts à chaque tag de version.

Points positifs

- L'architecture MVC a pu être respectée, de nouveaux contrôleurs ont pu être ajoutés à l'application afin d'étendre les fonctionnalités en gardant des fichiers lisibles et compréhensibles : `hotspot_controller.py`, `network_controller.py` et `server_controller.py` ont pu être ajoutés sans modifier le fonctionnement existant de l'application, conformément au principe de responsabilité unique.
- La décision de séparer `server_controller.py` de `app_controller.py` a permis de réutiliser directement la logique serveur dans le mode CLI sans duplication de code.
- La transparence sur l'utilisation de l'IA (commentaires `# AI USE` dans le code, mentions dans le journal de travail) garantit une traçabilité honnête des contributions de l'assistance de l'intelligence artificielle.
- Le pipeline CI/CD automatise entièrement le build, le packaging et la publication de l'application, ce qui permet d'accélérer le développement avoir un environnement plus prévisible pour travailler.

Difficultés particulières

La difficulté principale rencontrée a été un problème matériel : la fonctionnalité de point d'accès Wi-Fi repose sur `netsh wlan hostednetwork`, qui exige une carte

réseau supportant l'hébergement réseau. L'ordinateur fourni par le CPNV en était dépourvu, et plusieurs adaptateurs USB ont dû être testé avant de trouver un adaptateur compatible ce qui a retardé la validation de cette fonctionnalité de plus d'une semaine. Ce risque avait été identifié dans la matrice des risques (risque 1) et a été résolu grâce à l'aide reçue de M. Wulliamoz.

Une seconde difficulté a été le comportement différent entre l'environnement de développement et l'exécutable packagé. Plusieurs problèmes ont émergé et ont rendu la majorité des releases proposés lors du développement inutilisable ou incomplet. Voici les problèmes rencontrés :

- CWD au lancement depuis le menu contextuel
- Fenêtre de l'application s'ouvrant vide en mode CLI,
- Chemins de build incorrects

Expérience d'apprentissage

Ce projet a permis d'approfondir plusieurs compétences techniques : la programmation asynchrone avec ``asyncio`` et ``aiohttp``, la gestion des processus système sous Windows via ``subprocess`` et le registre Windows, ainsi que la mise en place d'un pipeline CI/CD complet avec GitHub Actions. La contrainte de développer une application Windows cible depuis un environnement Linux a également renforcé la rigueur dans la séparation entre le code de développement et le comportement en production.

Perspectives d'amélioration

Plusieurs pistes d'amélioration ont été identifiées pour une version future de Share++ :

- Ajout de la navigation dans les sous dossiers des répertoires partagés
- Ajout d'étapes de build d'OS différents (macOS, Linux, Android, IOS) afin d'étendre la compatibilité de l'application.
- Ajout de la détection Tailscale et la création du hotspot ad hoc pour la version CLI de l'application

5 Annexes

5.1 Résumé du rapport du TPI

Share++ Partage de fichiers en réseau local

Contexte et objectif

Share++ est une application Windows permettant le partage de fichiers et de répertoires entre les appareils d'un même réseau local, sans recours à des services tiers (cloud, e-mail, clé USB). L'utilisateur sélectionne un dossier, démarre un serveur de fichiers, et les autres appareils y accèdent depuis un navigateur web en scannant un QR code ou en saisissant l'URL générée automatiquement. Ce TPI est une évolution du projet préTPI (Share++ v1.0), auquel cinq axes d'amélioration ont été ajoutés : journalisation persistante, détection du service Tailscale, création d'un point d'accès Wi-Fi ad hoc, utilisation en ligne de commande, et refonte de la page de téléchargement.

Réalisation

L'application repose sur une architecture MVC en Python, avec Flet pour l'interface graphique et aiohttp pour le serveur HTTP asynchrone. La logique métier est décomposée en quatre contrôleurs indépendants (app_controller, server_controller, network_controller, hotspot_controller), ce qui a permis d'introduire chaque nouvelle fonctionnalité sans modifier le code existant. Un pipeline GitHub Actions génère et publie automatiquement deux artefacts à chaque version : un installateur Windows (SharePlusPlus_Setup.exe) et un exécutable CLI autonome (sharepp-cli.exe).

Tests et qualité

Les fonctionnalités principales sont couvertes par des tests unitaires et d'intégration automatisés avec pytest. Les tests End-to-End ont été réalisés manuellement avec un smartphone Android comme client. La principale difficulté matérielle était l'absence de carte Wi-Fi compatible avec le mode réseau hébergé Windows a et elle a été documentée comme contrainte externe et couverte par des tests unitaires avec appels système simulés.

Conclusion

Les objectifs principaux du cahier des charges ont été atteints. L'application est fonctionnelle, installable et utilisable dans un contexte réel, aussi bien pour un utilisateur grand public que pour un utilisateur avancé via l'interface CLI. Les perspectives d'évolution identifiées (navigation dans les sous-répertoires, compatibilité multiplateforme) sont documentées dans les conclusions du rapport.

5.2 Sources

Flet

Framework UI python

- <https://flet.dev/docs>
- <https://flet.dev/gallery>
- <https://flet.dev/docs/publish/windows/>

aiohttp

Serveur http

- <https://docs.aiohttp.org/en/stable/index.html>
- https://docs.aiohttp.org/en/stable/web_quickstart.html#run-a-simple-web-server
- https://docs.aiohttp.org/en/stable/web_advanced.html#middlewares

asyncio

Programmation asynchrone

- <https://docs.python.org/3/library/asyncio.html>

logging

- <https://docs.python.org/3/library/logging.html>
- <https://docs.python.org/3/library/logging.handlers.html>

subprocess

Execution de commande sur la machine

- <https://docs.python.org/3/library/subprocess.html>

socket

détection des adresses IP de la machine

- <https://docs.python.org/3/library/socket.html>

re

Expressions régulières

- <https://docs.python.org/3/library/re.html>

pathlib

Gestion des chemins de fichiers/répertoires

- <https://docs.python.org/3/library/re.html>

hashlib

Hash SHA-256

- <https://docs.python.org/3/library/hashlib.html>

qrcode

Génération des qr codes (GUI & CLI)

- <https://pypi.org/project/qrcode/>

shutil / tempfile

Compression .zip

- <https://docs.python.org/3/library/shutil.html>

- <https://docs.python.org/3/library/tempfile.html>

os

Actions générales sur le système

- <https://docs.python.org/3/library/sys.html>

sys

paramètres de l'interpréteur

- <https://docs.python.org/3/library/sys.html>

platform

Détection de l'OS

- <https://docs.python.org/3/library/platform.html>

ctypes

Appels windows (élévations admin)

- <https://docs.python.org/3/library/ctypes.html>

base64 / io

encodage des QR codes :

- <https://docs.python.org/3/library/base64.html>
- <https://docs.python.org/3/library/io.html>

webbrowser

ouverture dans le navigateur

- <https://docs.python.org/3/library/webbrowser.html>

datetime

horodatage des logs

- <https://docs.python.org/3/library/datetime.html>

argparse

interface CLI

- <https://docs.python.org/3/library/argparse.html>

getpass

saisie sécurisée du mot de passe

- <https://docs.python.org/3/library/getpass.html>

Tailscale

Plage adresses IP Tailscale et Tailscale CLI

- <https://tailscale.com/docs/concepts/tailscale-ip-addresses>
- <https://tailscale.com/docs/reference/tailscale-cli?tab=windows>
-

Workflow CI/CD

- https://dev.to/rahul_suryash/steps-to-build-binary-executables-for-python-code-with-github-actions-4k92

Documentation Python : logging, subprocess, socket, hashlib, pathlib :
<https://docs.python.org/3>

Documentation Tailscale : adresses IP 100.x : <https://tailscale.com/kb/1015/100.x-addresses>

Documentation PyInstaller : <https://pyinstaller.org/en/stable>

Articles et guides

(Guides Medium sur aiohttp)

(Autres références à compléter)

Utilisation de l'intelligence artificielle

Durant ce projet, des outils d'IA (Gemini) ont été utilisés ponctuellement comme aide au débogage et à la compréhension, principalement lors de problèmes bloquants : contraintes de privilèges Windows (CREATE_NO_WINDOW, élévation UAC), comportement différents entre l'environnement de développement et l'exécutable packagé, et stratégie de test pour les fonctionnalités dépendantes du matériel.

Dans tous les cas, le code proposé par l'IA a été relu, compris et adapté avant intégration. Aucun bloc de code n'a été copié-collé sans analyse préalable. Les contributions de l'IA sont systématiquement signalées par des commentaires # AI USE ou # AI FIX dans le code, et avec une description de la nature de l'aide apportée. L'utilisation de l'IA a également été mentionnée scrupuleusement dans le journal de travail.